Offline Real-Time Speech to Text

Windows 11 • Python 3.11 • Mandarin Chinese and English • Local processing

Purpose. A staged, testable implementation prompt for a private, offline-capable Windows speech-to-text app. Build only one stage at a time and wait for approval.

Project goal

Build a Windows desktop application in Python that continuously listens to the selected microphone and converts speech into text in real time. It must support Mandarin Chinese, English, and mixed Chinese-English speech. There is no wake word. Show partial text while the person is speaking, then commit a final sentence when the speaker pauses.

Priorities, in order: reliable live transcription, balanced speed and accuracy, low resource use, clear behavior, and privacy. The app must work locally after setup. Normal application use must not require an internet connection or make network requests.

Hard requirements

Platform: Windows 11, 64-bit Python 3.11.

Use a responsive Tkinter desktop interface. Keep microphone capture and recognition work off the Tkinter main thread.

Do not use cloud transcription services, remote APIs, telemetry, or background network requests. Audio must stay on the local machine.

Never write microphone audio to disk. Keep transcript history in memory unless the user explicitly presses Export. Logs must never contain audio or transcript text.

The Stop action must stop microphone capture promptly, release the input stream safely, and prevent stale audio from appearing after a later restart. Define and test how already queued audio is handled: preferably drain the queue for a short, bounded finalization period; otherwise discard it clearly and show a status message.

The user must be able to start and stop repeatedly without restarting the application. Handle missing microphones, device changes, model errors, and unexpected stream failures with readable messages.

Do not add dependencies beyond the approved list without asking first. Do not silently replace the selected recognition engine or model.

Approved technology stack

Core dependencies: sherpa-onnx, sounddevice, NumPy, and pytest. Use Python's standard library and Tkinter for the rest. Do not install PyTorch.

Primary recognition engine: sherpa-onnx streaming transducer recognition with endpoint detection. Initial model for the first benchmark: sherpa-onnx-streaming-zipformer-bilingual-zh-en-2023-02-20. Prefer the model's published int8 encoder and joiner files when the official package provides them, and use the matching decoder and tokens files from the same model release.

Treat this model as the initial baseline, not as a proven best model. Once the baseline works, compare it with one suitable alternative bilingual streaming model if available. Record the model source, exact file names, file sizes, and measured results. Do not change the baseline without documenting the comparison.

Optional Stage 10 only: faster-whisper with a local base model in int8 CPU mode to refine completed sentences. This stage is not required for the first usable version and must not delay the streaming baseline. Ask before adding its dependencies.

Working rules for the coding agent

Inspect the current workspace, Python installation, package versions, files, and microphone devices before making changes. Do not assume APIs, file names, paths, or device behavior. Check the installed package documentation, examples, signatures, or help output when uncertain.

Work on one stage at a time. At the end of each stage, list the files changed, explain what changed in plain English, run the relevant tests, show the actual command output or a faithful concise excerpt, identify remaining problems, and stop. Wait for the user to say “continue” before starting the next stage.

Pin dependency versions after verifying that the selected versions install and work together on Windows 11 with Python 3.11. Do not claim a test passed unless it was actually run. If a test cannot be run, explain why and provide the exact command for the user.

Do not overwrite unrelated files or delete user work. Keep code readable, typed where practical, documented, and split into small modules. Avoid premature optimization. Do not skip failing tests or hide errors.

Internet access is permitted only when the user explicitly runs setup or model download steps. The normal application must not make network requests. Do not add a package or feature without asking if it is outside the approved scope.

Project structure

Use this structure unless inspection finds an existing project layout that should be preserved:

```
speech2text/
  models/
  src/
    speech2text/
      __init__.py
      config.py
      logging_setup.py
      audio.py
      recognizer.py
      pipeline.py
      language.py
      gui.py
      main.py
  scripts/
    download_models.py
    check_setup.py
  tests/
  requirements.txt
  README.md
  .gitignore
```

Stage 0: Inspect and prepare

Check the Python version, pip version, working directory, available disk space, and microphone devices. Confirm Python is 64-bit and is version 3.11.

Inspect existing files before creating anything. Create or reuse a virtual environment, add an appropriate .gitignore, and install only the approved core dependencies: sherpa-onnx, sounddevice, NumPy, and pytest. Pin versions only after confirming compatibility.

Report the detected environment and any blockers. Do not begin Stage 1 until the user says “continue”.

Stage 1: Download and verify the model

Create scripts/download_models.py to download the chosen model only when the user explicitly runs the script. Use the official sherpa-onnx model release or documentation as the source. Explain the source URL and destination before downloading.

Extract the archive safely into models/. Print the actual encoder, decoder, joiner, and tokens file names found on disk. Do not guess or hardcode a filename that has not been verified. Save the verified relative paths in configuration.

If an int8 file is unavailable, report that fact and use a matching supported file only after confirming compatibility. Check that all required files exist and are non-empty. Do not download models automatically when the application starts.

Stage 2: Microphone capture

Implement audio capture with sounddevice.InputStream. The recognizer input should be mono, 16 kHz, float32 audio unless the verified model API requires another format. Use short blocks of approximately 100 ms.

Keep the audio callback minimal: copy the incoming samples and place them in a bounded queue. Do not run recognition or update the GUI from the callback. If the queue is full, drop the oldest block, increment a counter, and make the drop visible through status or diagnostics.

Check whether the selected microphone supports the requested sample rate. If it does not, choose a supported capture rate and resample correctly to the recognizer rate using an approved dependency or a verified standard-library-free implementation. Do not merely relabel samples with a different sample rate.

Add a short microphone test that captures about five seconds and reports sample rate, channel count, peak level, and whether the signal appears silent. Do not save the audio. Make stopping safe when called more than once, and release the stream even after an exception.

Stage 3: Streaming recognition

Wrap the verified sherpa-onnx API in recognizer.py. Confirm the installed version's actual constructor and method signatures before implementing. Load the model once and reuse it across recognition sessions where supported.

Enable endpoint detection. Start with documented endpoint settings near 2.4 seconds of trailing silence, 1.2 seconds for the shorter rule where applicable, and a maximum utterance length near 20 seconds, but verify the API and tune with real tests. Do not assume these settings map directly to the installed version.

Provide a feed(samples) interface that returns the latest partial text and an optional completed utterance. On an endpoint, commit text exactly once, reset or replace the stream as required by the API, and prevent duplicated words across adjacent utterances. Flush or finalize the last utterance on Stop when supported.

Add a terminal runner for testing before connecting the GUI. Test Mandarin, English, mixed Chinese-English speech, silence, short utterances, and longer utterances. Record real observations rather than claiming accuracy without a labelled test set.

Stage 4: Worker pipeline and queues

Use one recognition worker to consume audio blocks and send structured events to a thread-safe GUI queue. Suggested events: partial, final, status, warning, and error. Include timestamps for final utterances and processing duration where available.

Keep all Tkinter operations on the main thread. Use bounded queues and a documented overload policy. Count dropped audio blocks and warn the user if the worker falls behind. Do not create multiple recognizer workers for the same stream.

Define shutdown behavior explicitly. Stop accepting new microphone blocks first, close the stream promptly, then allow a short bounded period for queued audio to finish. If the deadline expires, discard remaining blocks and report that the last utterance may be incomplete. Clear old queues and reset recognizer state before a new session.

Stage 5: Tkinter interface

Build a simple, responsive Tkinter interface with Start and Stop buttons, microphone selection and refresh, language-label preference, status display, scrolling transcript history, a live partial-text line, Copy, Clear, and optional Export controls.

Show clear states such as Idle, Loading model, Listening, Stopping, and Error. Load the model on a background worker so the window does not freeze. Poll the GUI event queue with root.after at roughly 50 ms intervals. Only the main thread may update widgets.

Export transcripts only when the user explicitly chooses Export. Make clear that Export writes text, not audio. Copy should copy text to the clipboard. Clear should clear the visible in-memory transcript. Handle window close by stopping capture and releasing resources cleanly.

Stage 6: Language labels and text handling

Support Auto, Chinese, and English as display preferences. Auto should label each utterance using a documented heuristic. Chinese and English should control labels only unless the verified recognition model explicitly supports language forcing. Do not claim that changing the label forces the recognizer to understand only that language.

A basic Unicode CJK character check can help distinguish Chinese-containing text, but it is only a heuristic. Use a Mixed Chinese-English label when both Chinese characters and Latin letters are present. Document edge cases, such as English names inside Chinese sentences.

Trim unnecessary leading and trailing whitespace without rewriting recognized words. Avoid aggressive punctuation or text normalization that changes the recognizer output.

Stage 7: Configuration, logging, and setup checks

Use config.json for user-adjustable settings such as model paths, microphone selection, thread count, queue size, and endpoint tuning. Validate settings and show useful errors for invalid paths or values.

Set up rotating logs for technical diagnostics only. Never log microphone audio, transcript text, or copied/exported text. Do not include personal speech in exception messages.

Create scripts/check_setup.py to verify Python version, required imports, microphone access, model file presence, and actual model loading. Include a short local smoke test with a bundled or generated silent/test input where supported. Merely checking that model files exist is not enough to prove the recognizer can load them.

Stage 8: Automated and manual tests

Write unit tests using mocks for microphone capture and recognition. Cover silence, short and long utterances, queue overload, dropped-block counts, microphone errors, missing model files, clean shutdown, GUI event delivery, restart after Stop, and the rule that audio is never written to disk.

Keep real microphone and model tests separate from fast unit tests. Provide clear commands for both. Use two short user-provided WAV clips for integration tests, one Mandarin and one English or mixed-language clip, if the user supplies them. Do not invent test results or silently skip failures.

Verify that partial text updates while speaking, final text appears after a pause, each utterance is committed once, the interface remains responsive, and repeated Start/Stop cycles work.

Stage 9: Offline and performance verification

After the model and dependencies are installed, disconnect the network and test normal application use. Confirm the application starts, loads the local model, listens, transcribes, and exports without internet access. Review the code to confirm that normal application modules do not make network requests; only explicitly run setup or model download scripts may do so.

Measure end-of-speech-to-final-text delay, partial-text update delay, average and peak CPU use, peak RAM, dropped audio blocks, and real-time factor (RTF). Calculate RTF as recognition processing time divided by the duration of the audio processed. RTF below 1 means the processing ran faster than the audio duration, but endpoint silence and scheduling can still affect the user's perceived delay.

Compare thread counts of 1, 2, and 4 when supported. Compare int8 and non-int8 model files only when both are compatible and available. Record hardware, model filenames, sample conditions, measurements, and observed transcription errors. Do not call a model more accurate without a consistent test set and reference transcripts.

If feasible, compare the baseline with one alternative bilingual streaming model. Keep the baseline as the default unless measured results justify a change. Document the offline test procedure and results in README.md.

Stage 10: Optional final-sentence refinement

Do not begin this stage until Stages 0 through 9 are complete and the user explicitly approves the added dependencies. Optionally use faster-whisper with a local base model in int8 CPU mode to re-check completed sentences while sherpa-onnx continues to provide live partial text.

Run refinement in one background worker. Keep audio in memory only, do not save it, and do not make network requests. Replace a final sentence only if the refinement result is clearly aligned with that utterance and improves it. Avoid duplicate output and keep a setting to disable refinement.

Benchmark the effect on final delay, CPU, RAM, and recognition quality. If it does not provide a measurable benefit, leave it disabled or remove the optional stage.

README and final deliverables

The README must include supported Windows and Python versions, setup commands, explicit model download steps, how to start the application, how to select a microphone, how to use the language-label preference, how to stop and export, how to troubleshoot common errors, how to run tests, and how to verify offline operation.

Document known limitations, the exact model and dependency versions tested, how endpoint settings affect responsiveness, the meaning of language labels, and measured performance on the available machine. State clearly that recognition quality varies with microphone quality, accent, noise, speaking rate, and code-switching.

The project is complete when the app shows partial text during speech, commits final text after a pause, supports Mandarin, English, and mixed speech, stays responsive, handles errors and repeated Start/Stop cycles, writes no audio files, exports text only on request, works offline after setup, and has meaningful tests with reported results.

Required response after every stage

- Stage completed:
- Files changed:
- What changed:
- Commands run and actual results:
- Tests passed and tests failed:
- Known problems or limitations:
- Next stage waiting for user approval:

Stop after reporting the current stage. Do not proceed until the user says “continue”.

Start instruction

Begin with Stage 0 only. Inspect the environment and existing workspace, report findings, prepare only what Stage 0 requires, run the relevant checks, and then stop for approval. Do not implement Stage 1 or any later stage until the user says “continue”.